



Digital Design Verification

Lab Manual # 25 – Cache Architecture Exploration

Release: 1.0

Date: 2-July-2024

NUST Chip Design Centre (NCDC), Islamabad, Pakistan



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.

Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
1.0	2/07/2024	Ali Aqdas	Complete manual	-



Contents

Objective.....	3
Tools and Technologies.....	3
Submission Guidelines.....	3
Overview.....	4
Task 1: Random Replacement Policy.....	5
Task 2: Least Recently Used Policy	5
Investigation Questions.....	5
Q1. Evaluation for Random Replacement Policy.....	5
Q2. Evaluation for Least Recently Used Policy	5
Reference	5



Objective

The objective of this lab is to

- Understand the Cache Functionality and Implement Least Recently Used (LRU) Overwrite Policy

Tools and Technologies

- Cache Simulator
- C/C++
- GNU Toolchain

Submission Guidelines

You will need to submit the updated emulator files and a single report, answering the questions listed in the following section.



Overview

In this lab, you will be implementing various cache eviction policies in a software cache simulator available on [GitHub](#).

The cache relevant code is in the **cachesim.cpp** and **cachesim.h** files. At the top of **cachesim.h**, the number of sets, ways, and size of a cache line (in words) are specified via **#defines**. In **cachesim.cpp**, the relevant functions are **cache_read**, **cache_write**, and **cache_flush**.

The **cache_flush** function chooses a cache way to evict, flushes the data into memory, and then clears the valid bit of the flag. Right now, the **cache_flush** function always chooses way 0 to evict, which is not very effective. Your task is to implement LRU and measure performance.

Two small benchmarks are provided

- **example_questions/sort.s** implements quicksort on 1024 elements
- **example_questions/graph.s** implements single-source shortest path on a graph with 64 nodes

Performance can be measured via the number of words flushed from the cache, as seen in the following figure:

```
Reached Halt and Catch Fire instruction!
inst: 170475 pc: 40 src line: 16
x00:0x00000000 x01:0x00000028 x02:0x0000ffff x03:0x00000000 x04:0x00000000 x05:0x00000000 x06:0x00000000 x07:0x00000000
x08:0x00010000 x09:0x00000000 x10:0x00000000 x11:0x00000000 x12:0x00000000 x13:0x00004660 x14:0x00000040 x15:0x0000003f
x16:0x00000000 x17:0x00000000 x18:0x00000000 x19:0x00000000 x20:0x00000000 x21:0x00000000 x22:0x00000000 x23:0x00000000
x24:0x00000000 x25:0x00000000 x26:0x00000000 x27:0x00000000 x28:0x00000000 x29:0x00000000 x30:0x00000000 x31:0x00000000
Cache read 37260/42126 Cache write 11088/15681
Cache flush words: 9459
Execution done!
```

Cache flush words display the number of words flushed from the cache

The description of remaining would be as follows

Cache Read: <Cache Read Hits>/<Read Requests>

Cache Write: <Cache Write Hits>/<Write Requests>



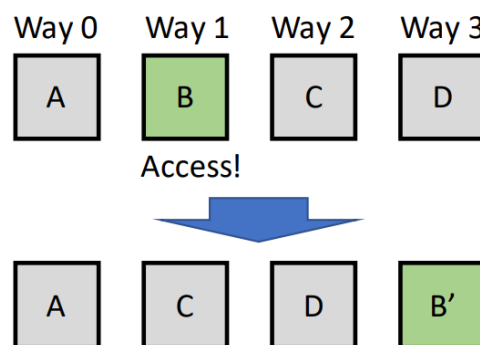
Task 1: Random Replacement Policy

Try a random replacement policy by changing the **cache_flush** function to select

```
way = rand() % CACHE_WAYS
```

Task 2: Least Recently Used Policy

The most straightforward implementation would be augment **cache_read** and **cache_write** to move the accessed <cache_line,flag,tag> values to the last way, shifting everything down by one slot. This way, the least recently used cache line will always be in way 0. The following figure illustrates this



Investigation Questions

Q1. Evaluation for Random Replacement Policy

Given a budget of 256 words in the cache (e.g., 256 sets, 1 way, 1 word per cache line, or 64 sets, 2 way, 2 words per cache line), what is the best set/way/line configuration for the two benchmarks? Is there a difference, why?

Remember, the cache configuration parameters are in **cachesim.h**, and are given in logarithmic terms. E.g., **CACHE_SETS_SZ "8"** means the cache has 2^8 sets. Do not modify **CACHE_SETS** and similar defines directly, as it may break the code.

Q2. Evaluation for Least Recently Used Policy

Same as with the random replacement policy, what is the best configuration, and is there a difference between the two benchmarks, and why?

Reference

Cache Architecture Exploration (Lab 3) [CS152 (UC Irvine)]